RESEARCH-ARTICLE

Adaptive Compression for Fast Scans on String Columns

YANNIS FOUFOULAS, Athena - Research and Innovation Center in Information, Communication and Knowledge Technologies, Athens, Attica, Greece

LEFTERIS SIDIROURGOS, National and Kapodistrian University of Athens, Athens, Attica, Greece

ELEFTHERIOS STAMATOGIANNAKIS, National and Kapodistrian University of Athens, Athens, Attica, Greece

YANNIS IOANNIDIS, Athena - Research and Innovation Center in Information, Communication and Knowledge Technologies, Athens, Attica, Greece

Open Access Support provided by:

National and Kapodistrian University of Athens

Athena - Research and Innovation Center in Information, Communication and Knowledge Technologies

Published: 18 June 2021

Citation in BibTeX format

SIGMOD/PODS '21: International Conference on Management of Data
June 20 - 25, 2021
Virtual Event, China

Conference Sponsors:
SIGMOD

https://doi.org/10.1145/3448016.3452798
ISBN: 9781450383431

Adaptive Compression for Fast Scans on String Columns

Yannis Foufoulas
University of Athens, Greece
Athena Research Center
johnfouf@di.uoa.gr

Lefteris Sidirourgos
University of Athens
Greece
lsidir@di.uoa.gr

Eleftherios Stamatogiannakis
University of Athens
Greece
estama@gmail.com

Yannis Ioannidis
Athena Research Center
University of Athens, Greece
yannis@di.uoa.gr

ABSTRACT

State-of-the-art OLAP systems tend to use columnar data representations, as these are both suitable for analytics and amenable to compression. Local dictionary value encoding has been shown to achieve high compression rates for string columns while still allowing fast filtered scans. In this paper, we argue that the effectiveness and efficiency of local dictionary compression is limited by data repetition across file blocks and by dictionary look-ups inside each block during filtered scan execution. To address this problem, we introduce an adaptive compression technique that is based on differential dictionaries and targets both storage efficiency and query performance. The proposed scheme reduces dramatically the need to store repeated values across different file blocks and significantly accelerates read operations by reducing the time needed for dictionary look-ups. A preliminary set of experiments has given very promising results, showing that, in many cases, the proposed new dictionary compression scheme is much more efficient than existing techniques, occasionally up to an order of magnitude.

ACM Reference Format:
Yannis Foufoulas, Lefteris Sidirourgos, Eleftherios Stamatogiannakis, and Yannis Ioannidis. 2021. Adaptive Compression for Fast Scans on String Columns. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21), June 20–25, 2021, Virtual Event, China.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3448016.3452798

1 INTRODUCTION

Since data volumes processed by distributed applications are growing rapidly, new bottlenecks have emerged. One of those regards the data transfer and processing cost. In distributed OLAP, huge amounts of data are transferred across the network. Data movement has an important impact on the execution time of analytical processing tasks. Obviously, data movement is affected by the size of the data. So, if we want to reduce it, it is important to reduce the size of the data. Processing cost is usually the cost to run a scan, filtered scan or a join and is affected by the data layout.

OLAP systems use DSM [14] or Hybrid formats [15], which are amenable to compression and fast in scans. These formats use a combination of compression techniques (e.g., RLE, bit packing, byte compression like SNAPPY [1], ZSTD [2]). One of the most prominent compression technique is dictionary based compression. There are two main approaches to dictionary encode an attribute: *1)* local (block-level) dictionary encoding, and *2)* global (table-level) dictionary encoding. To compare these approaches we take into account how they perform in the following operations: *i)* Compression, *ii)* query processing and *iii)* random access.

Global dictionaries reduce the size for storing the actual values, since it does not store repeating values across the blocks. This makes a difference in several cases, however, a block compressed with a local dictionary might be smaller. A local dictionary contains fewer values than a global dictionary, so a value compressed with a local dictionary can be represented by fewer number of bits or bytes. Additionally, keeping a global dictionary may be inefficient in terms of memory consumption when the column's cardinality is high.

As for query processing, local and global dictionaries use block level statistics to filter out blocks during a scan. Local dictionaries require dictionary look-ups per block but they scan the offsets only in the necessary blocks whereas global dictionaries require one look-up but they scan all the offsets unless the block is filtered out. In random access, local dictionary encoding is faster since there is no need in looking up a big global dictionary.

The block size is also an important parameter. Small blocks lead to lower compression rate while big blocks mean higher memory footprint and expensive random access [5]. Since compression rate is very important because of network communication the increased block size (usually more than 256MB) is mostly selected [5, 6, 10].

In this work, we introduce adaptive dictionary compression which balances between local and global encoding, exploits the benefits of both methods and achieves high compression rates while storing small pages. Our method is inspired

SIGMOD '21, June 20–25, 2021, Virtual Event, China

ACM ISBN 978-1-4503-8343-1/21/06…$15.00
https://doi.org/10.1145/3448016.3452798

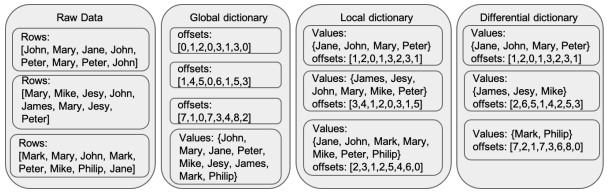

Figure 1: Different dictionary formats

by video compression with I and P frames [31]. In video compression, I frames are the least compressible but do not require other video frames to decode. P frames use data from previous frames to decompress and are more compressible than I frames. Similarly, we aim at separating data blocks to least compressible and very compressible but dependent on previous blocks. The proposed format consists of local and self-contained dictionaries and differential dictionaries which contain new and different values to the previous dictionary [1]. This method reduces the storage of repeating values across blocks, thus leading to higher compression rates. Figure 1 presents an example visualization of the dictionary encodings with some string values. Encoding all blocks with differential dictionaries can be considered as an alternative incremental implementation of global dictionaries and involves many of its limitations (i.e., offset size, dictionary length). So, we propose an adaptive dictionary encoding technique which smartly selects between a local and a differential dictionary, aiming at the right balance between local and global encoding.

The main contributions of this work are the following:

- The differential dictionaries that lead to better storage/scan efficiency than other dictionaries.
- The cost function that is used to select between a local and a differential dictionary adaptively.
- The different implementations of the proposed method (Dask[19], C++) that prove its validity.

2 BACKGROUND

Several compressed formats have been implemented. Record Columnar File (RCFile) [22] is based on PAX [15], and compresses each column with ZLIB [3]. ORCFile [21] is data type aware and uses local dictionary encoding for low-cardinality columns and large blocks (default 250MB). ORC File stores small indices (min/max, count, etc.) to optimize query evaluation. PARQUET [25] uses dictionary encoding per column. In PARQUET, data are grouped in horizontal partitions called row groups. Each row group contains a column chunk for each column and compresses at page level.

IBM DB2 [17, 20, 24] uses row compression with a global dictionary. With BLU acceleration [16, 34] it employs columnar dictionary compression based on the frequency of the values, so that the most frequent values are assigned the smallest codes. SAP HANA [35] and Data Blocks [27] target both OLAP and OLTP. SAP HANA implements a unified table structure which allows both updates using a row store and scans with a column representation compressed with sorted

[1] Available on GitHub: https://github.com/madgik/arcade

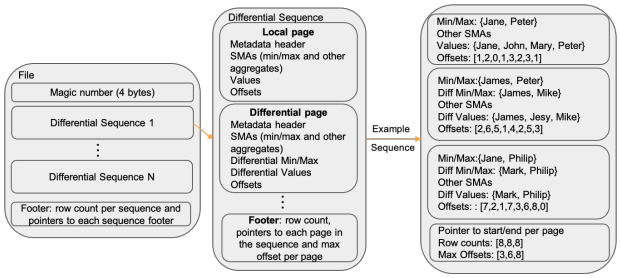

Figure 2: Structure of an adaptively encoded attribute

dictionary encoding per page and bit-packing. Data Blocks [27] uses dictionary encoding per block and selects the appropriate compression with regard to the memory consumption while also uses Positional SMAs (PSMAs) to optimise scans.

Abadi et al. [13] implemented dictionary encoding per column. According to the attribute cardinality, they decide whether values should be packed into 1, 2, 3, or 4 bytes for cache performance. Indirect coding [29] introduces a mapping from the stored local offsets of a block to the offsets of the global dictionary. It is enabled when a block contains a few distinct values so that the block size is not increased by the storage of the map. With this mapping it keeps storing small offsets but it requires one extra step to decode a value.

Ordered global dictionaries are efficient in range queries as they run without decoding the values. However, creating an order-preserving global dictionary requires the full set of the values known a priori. Binnig et al. [18] introduce data structures to support order-preserving dictionary compression. Liu et al. [30] present mostly order-preserving (MOPs) dictionaries. They pre-allocate space based on estimated statistics to store a MOP and support range queries without decoding the values that exist in the MOPs.

Zukowski et al. [37] proposes patched dictionary compression (PDICT). PDICT uses local dictionary encoding per chunk and encodes only the frequent values. When the set of the frequent values remains the same across the chunks it reuses the dictionary. This format is mainly beneficial with highly skewed datasets with a small set of values which are very frequent across all the blocks of the file.

BRPFC [28] targets efficient compression on sorted string dictionaries and Müller et al. [33] proposes a prediction method to select the optimal among several dictionary compression schemes (e.g., Huffman / Hu-Tucker Compression [26], Bit Compression, N-grams, Front Coding [36] etc.).

3 COMPRESSION AND PROCESSING

Figure 2 depicts the structure of an adaptively compressed attribute and an example of a differential sequence using the same data as in Figure 1. Each attribute consists of multiple differential sequences. Each sequence consists of a locally encoded page and N pages encoded with differential dictionaries. Each page contains the values, the offsets, small aggregates like SMAs [32] and metadata (i.e., dictionary length, offset size, pointers). A differential page stores only the newly added values in a sorted list and its min/max values.

So, with differential dictionaries the storage of the repeating values is eliminated and the time spent for dictionary look-ups which are sorted and smaller than local dictionaries is reduced. The offsets are stored with integer types according to the dictionary length. Similarly to the other formats, dictionary encoding is disabled for high cardinality columns according to a threshold (max rate of distinct values per page: 80%). Each page contains up to 2^{16} records so that the offsets do not require more than 2 bytes. Maintaining small pages leads to low memory footprint and better cache performance, which is crucial specifically in the case of complex tasks with several writers getting involved. The same number of records is also proposed by Data Blocks [27] while ORCFile proposes a large stripe size (250MB) and a memory manager [23] to reduce the stripe size when the memory is limited. However, each stripe includes much smaller dictionary encoded row index strides (10000 records). The footer of a differential sequence contains pointers to each page, row counts, and a list containing the maximum offset for each page in sequence. An attribute may consists only of locally encoded pages or of just one local dictionary and all the rest encoded differentially. Employing only local dictionaries suffers from two main limitations:

- Look-ups take place in each block during the execution of a query while in case of global dictionaries this happens just once but on a bigger dictionary.
- Elimination of duplicate values is applied within a block so values are repeated across different blocks.

Adaptive encoding balances between local and global encodings combining the benefits of both techniques. The presented encoding is called "adaptive" because it selects between a local or a differential dictionary at compression time according to a cost function. The cost function (section 3.1) adapts to the characteristics of the attribute and decides how many pages will be encoded with differential dictionaries in each sequence. So, it is possible that the cost function may select only local dictionaries across all the pages of the attribute (e.g., when there are almost no repeating values across the pages), or simulate global dictionaries (e.g., when the total count of distinct values is small enough) by using only differential dictionaries. An attribute encoded with adaptive dictionaries is splittable at the end of a sequence. This is analogous to other formats (e.g., Parquet, ORC) which are splittable at the end of a row group. Since differential sequences are variable sized, to support parallelism, the compressed file consists of partitions with a user-defined size (e.g., 1GB) where each partition is encoded with adaptive dictionaries and contains multiple differential sequences.

The format allows compression on its sorted dictionaries or offsets using codecs like ZLIB [3], RLE/Bit-Packing Hybrid or techniques like [28] and [33] mentioned in background section. Moreover, techniques like PDICT [37] or BLU Acceleration [34] differ from the one proposed in several aspects: *1)* PDICT does not encode the rare values in a chunk. However, rare values in one chunk may become more frequent in subsequent chunks. Our technique does not make such a local decision but encodes all values and reuses offsets throughout. *2)* PDICT and BLU Acceleration require samples and frequency histograms, while the proposed technique adapts at compression time to the current distribution and compresses passing over the data once. *3)* PDICT encodes only the frequent values and BLU Acceleration sorts the dictionary by frequency to assign small codes to the most frequent values, thus they are more effective for highly skewed datasets, while the proposed technique adapts on all types of distributions (as shown in the experiment section). *4)* PDICT is based on local dictionaries, with the occasional reuse of one in the next chunk in specific cases of highly skewed datasets (e.g., if they share the same set of frequent values), and BLU Acceleration is based on global dictionaries. In contrast, our cost function dynamically balances between local and global dictionaries storing the differences and using offsets from previous dictionaries for close to optimal compression. With adaptive dictionary encoding, the compressed file will be smaller than techniques using local and global dictionary encoding since:

- The duplicates across pages are reduced.
- The size of offsets is equal to that of a local dictionary unless the cost function predicts that longer offsets lead to higher compression, and smaller than that of a global dictionary, which captures all values in file.

3.1 Selection between differential and local

The selection between a differential and local dictionary is the most important step of the proposed technique. The obvious metric is to maintain the offsets size as small as possible. So, when the number of new added values requires increasing the offsets' size, a local dictionary is used so that the offsets remain small. However, when the values length is big, the increase of the offsets' size may be preferable. Given this, the selection between a differential and a local dictionary should depend on the comparison of the required page size if it is locally or differentially encoded. However, the selection between a differential or a local dictionary also affects the next pages. A differential dictionary that comes after a local uses smaller offsets but it usually contains more values than a differential dictionary that comes after another differential dictionary. So, to make an optimal selection we have to consider what will happen in the next pages.

Figure 3 presents an example of 6 pages and what happens if a local or a differential dictionary is selected when the offset size increases. If the algorithm selects to store a differential dictionary, the current page is stored using less space, but the next pages require cumulatevily more space.

In order to deal with these issues, the algorithm selects between a differential and a local dictionary, using knowledge obtained from the previous sequence to predict what will happen in the next pages. We use every time the last sequence of dictionaries so that we adapt to changes that may happen over time to the statistical properties of the attribute.

Equation 1 shows how the statistics from previous pages are utilized to select the appropriate encoding. Differential dictionaries are preferable as they do not increase the offset size, so the following cost function is not evaluated in each

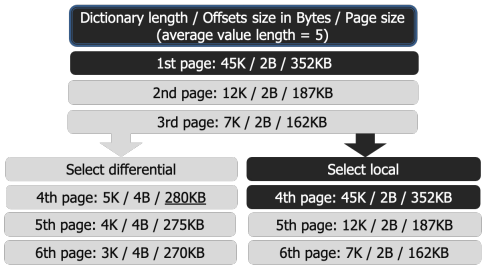

Figure 3: Differential vs local selection

page but only when the storage of a new differential dictionary increases the offsets' size. When the equation is valid, a local dictionary is stored, otherwise, the algorithm keeps on encoding the attribute with differential dictionaries.

$$\text{diffcount} * (\text{difOffsetSize} - \text{diffavg}) - \text{locOffsetSize} - (\text{locPageSize} - \text{difPageSize}) > 0 \quad (1)$$

difOffsetSize ← total offsets size for a differential dictionary
locOffsetSize ← size to store the offsets of a local dictionary
diffavg ← average size of a differential dictionary
diffcount ← count of differential dictionaries in sequence
locPageSize ← page size in case of a local dictionary
difPageSize ← page size in case of a differential dictionary

This cost function considers the size of the current page if it is local or differential and compares it with the prediction for the next pages. The prediction is made using the size of the differential dictionaries in the current sequence (assuming that the case will be similar for the next sequence if we select a local dictionary) and the required bytes to store the offsets for a local or a differential page. Applying equation 1 on the example in Figure 3, a local dictionary will be selected at the fourth page. This equation targets higher compression rates because we regard data transfer as the major bottleneck, however, as shown in the experiments, in most cases the smaller file also leads to faster scan/filter times.

Compression performance is important when a dataset is compressed online and then gets processed. Compression performance of differential dictionaries is mainly affected by the calculation of distinct values and different values across the pages. This gets bigger when each page contains many new distinct values. The algorithm considers the following: 1) the total size of the distinct values of the sequence. These values are kept in memory and should not exceed the size of cache since this could result in excessive memory consumption and slow calculation; and 2) the ratio of new value occurrences. Differential dictionaries are useful when there are many duplicate values across the pages. If during the calculation of the new values it seems that there is small value repetition (less than 10% by default), then the compression gains may be insignificant compared to the required calculation time. In this case, the algorithm stops using differential dictionaries and stores only local ones. To adapt to changes that may happen to the statistics, while the algorithm resorts to local dictionaries we need to fast check whether the rate of new values over the total values of a new page has exceeded the threshold. We create a bloom filter for a local dictionary encoded page and while processing the next pages we get an approximate count of the repeating values, so that we resort to differential dictionaries if the ratio increases.

Algorithm 1 Adaptive Dictionary Encoding

```
dictionaries = {}
for col in data_page:
    vals = distinct_vals(col)
    #diff contains values that do not exist
    #in any previous page of the sequence
    diff = vals - dictionaries[col]
    cost_function(local_dict, diff_dict)
    if local_dict:
        vals = sorted(vals)
        dictionaries[col] = vals
        store(vals)
    if diff_dict:
        diff = sorted(diff)
        dictionaries[col].append(diff)
        store(diff)
    calculate_and_store_the_offsets()
```

3.2 Explanation of implemented algorithms

3.2.1 Compression. Algorithm 1 shows the main steps of the proposed adaptive dictionary encoding. The algorithm maintains a dictionary that contains a list for each column in the table. When a new differentially encoded page is processed its values are sorted and appended to the end of the dictionary. When the cost function selects to store a local dictionary, the dictionary is not updated any more, but it is initialized with the sorted values of the current page.

3.2.2 Full Scan/Decompression. The full scan of a local dictionary is quite simple. For each page, the algorithm scans the offsets and replaces them with the actual values stored in the dictionaries. In case of differential dictionaries, the algorithm *i)* appends the list containing the new additional values to the previous list, *ii)* scans the offsets and replaces them with the actual values, and *iii)* reconstructs the rows.

3.2.3 Filtered Scan. The algorithm scans with filter, as shown in Algorithm 2. For each page, if the requested attribute is locally encoded the min and max value of the attribute are checked and the page is omitted if possible. Otherwise, the algorithm searches the sorted list of values and the offset of the requested value is returned if it exists. In case of a differential dictionary, if the value was already found in a previous page its offset is retrieved after checking the min/max value (this offset is stored in 'valid_offsets' dictionary in Algorithm 2). Otherwise, the differential min/max values are checked to omit the whole page before the sorted differential list is searched. If the value exists, its offset is retrieved and then the offsets are scanned to get the row identifiers. Contrary to compression, during the scan we do not need to keep and

Algorithm 2 Equi Filtered Scan

```
cur_offsets, valid_offsets = {}, {}
for col in data_page:
  if local_dict:
    dict = read_dict(col)
    cur_offsets[col] = len(dict)
    if minmax(col):
      valid_offsets = search(dict)
      if valid_offset[col]:
        offsets = read_offsets(col)
        row_ids = scan_offsets()
  if differential_dict:
    if valid_offsets[col]:
      if minmax(col):
        offsets = read_offsets(col)
        row_ids = scan_offsets()
    else:
      dict = read_dict(col)
      if diff_minmax(col):
        valid_offsets[col] =
        search(dict)+cur_offsets[col]
        cur_offsets[col] += len(dict)
        if valid_offsets[col]:
          offsets = read_offsets(col)
          row_ids = scan_offsets()
```

update the whole global dictionary. We just need to keep the last offset from the previous page. This method reduces the time needed for dictionary look-ups for the following reasons:

- Differential dictionaries are shorter (or even empty in many cases) than the local ones, so the time needed for dictionary look-ups is reduced.
- Differential dictionaries benefit from their tighter min/max and omit more pages in their entirety, compared with local and global dictionaries.
- Unlike global dictionaries, in an equi-filter, when the value is found in one page's dictionary, the next dictionaries in the sequence are entirely skipped and offsets are scanned directly. Similarly, offsets' scan is avoided until the first occurrence of the value in the sequence.

3.2.4 Page omittance. Using additional min/max values calculated over the new values of a differential page, we reduce significantly the false positives compared to zone maps, thus more pages are omitted. If a value is not found previously, then checking the tighter min/max values of the differential dictionary is enough. Adaptive dictionaries also avoid offsets scan until the first occurrence of the searched value in each sequence, whereas in global dictionaries if the value exists in the global dictionary all offsets are scanned (except when omitting a page using zone maps).

Differential min/max statistics require accessing the dictionary of a page and when the value is not found then check the differential min/max of the next page. Usually, zone-maps are stored in separate files for faster page skipping and used

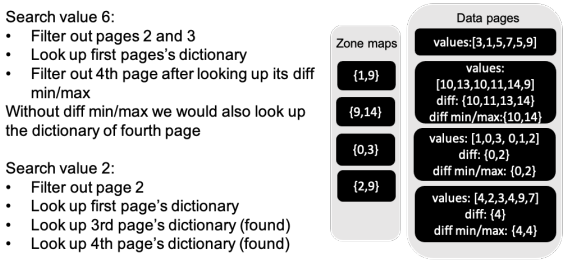

Figure 4: Combination of zone maps and differential min/max

to identify the candidate pages. When a page with a differential dictionary is identified, either it contains the value in its dictionary or it gets the value's offset from a previous dictionary which was also not omitted. While searching the dictionaries of identified pages, differential min/max statistics are used ignoring the omitted pages in between. Figure 4 depicts how differential min/max statistics work combined with zone maps. The example file consists of 4 pages, in each page we present the raw values, the new values that consist the differential dictionary, and the differential min/max. Zone-maps are stored separately. When searching for value 6, we omit pages 2 and 3 using the zone maps. Then, we look up the dictionary of the first page and since the value is not found, we look up the differential min/max of page 4 and omit the page. Since value 6 was not found in the dictionary of the first page, it should exist in the new added values of the current page, so we can use the tighter differential min/max instead of the zone map.

3.2.5 Random access. Random data accesses run in a fixed number of steps. Independently of the number of differential dictionaries in sequence, we always need to read up to 2 pages to decode a random row. The first page contains the corresponding row to get the offset. Its header contains a list with the size of each previous dictionary in the sequence. Empty previous dictionaries are not included in this list for storage efficiency. Given that during the encoding phase a new differential dictionary is "appended" (and not merged) to the previous, the offset and the dictionary sizes are enough to calculate and access directly the second page with the differential dictionary that contains the offset's value.

4 EXPERIMENTS

We implemented adaptive dictionaries and other dictionary formats in C++ and Python to compare with their performance. The Python implementation has also been integrated within Dask's Parquet [4] so that we compare scan queries with Parquet and its enhanced version with adaptive dictionaries. We ran experiments to examine time and space efficiency of adaptive dictionary encoding. We experiment with local, adaptive, global, and indirect coding. We run experiments on EDGAR Log File Dataset [7], TPC-DS [11], Microsoft Academic Graph [8], Yelp [12], OpenAIRE [9]. The experiments ran on an Intel Core i7-4790 processor with a 500GB SSD and 16GB RAM, running Ubuntu Server 18.04 LTS.

4.1 Experiments with various data distributions

For this experiment, we created synthetic data based on Microsoft Academic Graph. We created tables with affiliation names that follow zipfian distributions with various skewness parameter θ. A smaller θ means that the distribution is less skewed, so when $\theta = 0$, we have uniform distribution. The affiliation names contain on average 30 characters and the size of the attribute is 4.7GB when $\theta = 0$. For each distribution, we ran experiments with sorted and randomly ordered values.

We compare C++ implementations of local, adaptive, global dictionaries and indirect coding in terms of storage, scanning cost and random access. Global dictionaries are unsorted and implemented incrementally, so that even if their length is bigger than 2^{16} they store offsets with 1 or 2 bytes until the dictionary length exceeds the thresholds of 2^8 or 2^{16}. The queries ran in cold caches.

The first experiment regards the compression time, rate and the full scan/materialization of a table that consists of 120 million rows with 70K affiliation names. Figures 5 (a) and (b) present the results. The chart shows full size and dictionaries size (in transparent color). In most cases with global and adaptive, dictionaries size is negligible. As shown, with randomly ordered values and a small θ, local dictionaries compress worse than global encoding. This happens because global and adaptive encoding avoid storing repeating values, and given an average length of 30 bytes per value, value repetitions are more important than offsets size. In case of larger θ, there are many pages with a few distinct values, thus there are many pages in which the offsets size is smaller and local dictionary encoding performs better. Adaptive dictionaries perform even better than that, striking the balance between the value repetition and the size of the offsets. As shown when $\theta = 1.3$, adaptive dictionaries use some more space for dictionaries to reduce offsets size and total size. Indirect coding achieves high compression rates (i.e., for $\theta = 1.3$ it is a little worse than adaptive) but the creation of the mapping between local and global offsets leads to higher encoding time. Local and adaptive differential dictionary encoding perform better with sorted values. Each page contains less distinct values, so fewer bytes are used to store the offsets. The adaptive encoding uses mainly local dictionaries.

As for the full scan efficiency, adaptive dictionaries perform similarly or better than the other encodings depending on the data distribution. To evaluate filtered scan efficiency we executed a query that scans the affiliation attribute with a random value. The searched value exists 227 times when θ=1.3, 2200 times when θ=0.7, and 1670 times with θ=0. The results are shown in Figure 5 (a). Local encoding performs better when the values are sorted while global dictionaries are more efficient with randomly ordered values. Our adaptive format performs well with all distributions. It is faster than local in randomly ordered values because dictionary lookups are fewer and differential dictionaries are smaller than local. It is faster than global in highly skewed data and in sorted data since it reduces I/O and it is also faster than indirect coding which requires an extra step to decode each value.

format	total I/O	offset scan	dict look up
adaptive	418	57	48
global	800	62	0.036
local	1017	7.4	270
indirect	488	60	163

Table 1: Partial execution times for filtered scan

Table 1 shows the partial execution times in milliseconds for the randomly ordered skewed distribution. Global encoding requires more time for reading the offsets (it reads the dictionary in 14msecs). It accesses all the offsets (which are bigger than in adaptive encoding) while adaptive encoding reads the offsets only when the value is found in a dictionary of the sequence. Local encoding suffers from I/O and dictionary look ups but it is fast at offsets scan since this happens only after local dictionary look-up. Indirect coding requires more time for look-ups than global because of the mappings. It also requires a little more I/O than adaptive since it is a little bigger and it also reads all the offsets. While indirect coding is compatible with the proposed format, the overheads that the offsets mapping introduce both in encoding and processing time makes it not an ideal selection.

To evaluate random access queries, we selected randomly 10 row numbers, and run point queries. Figure 6 contains the average execution times. The main overhead in random access queries is the cost to load and access the dictionary. Global encoding contain one big dictionary so it is inefficient in random access queries. Indirect coding has the extra overhead of looking up the mapping for indirect coded pages but in case of a single random access this is exactly one small lookup and it increases the execution time less than 1 msec.

Comparing local and adaptive dictionaries in terms of random access several cases occurred: *i)* If the page in adaptive encoding is encoded locally, random access time is same to local dictionaries. *ii)* If the page is differentially encoded and the value exists in the dictionary current dictionary, adaptive dictionaries are faster because differential dictionaries are smaller than the local ones. *iii)* If the value exists in the local dictionary of the sequence, adaptive encoding has the overhead of one more disk access. *iv)* If the value exists in a previous differential dictionary, the overhead of the disk access is usually overcome by the loading of a differential dictionary which is smaller than a local, thus adaptive encoding may be faster than local in these cases.

For instance, in the uniform distribution when we access a row in the 15th page, we spent 19msecs to I/O with local dictionaries. This contains the access to the offsets and the local dictionary (I/O and deserialization of 43K values). With differential dictionaries, we access the offsets in the 15th page and encounter 3 cases: *i)* The offset exists in the differential dictionary of the third page (7K values). The I/O and deserialization time in this case is 8msecs. *ii)* The offset exists in the differential dictionary of the first page (43K values). The I/O time in this case was 26msecs and includes reading the page header, the offsets, the list with 15 short integers that contain the number of offsets in each previous page, and the dictionary from the 1st page. Since the list with the previous

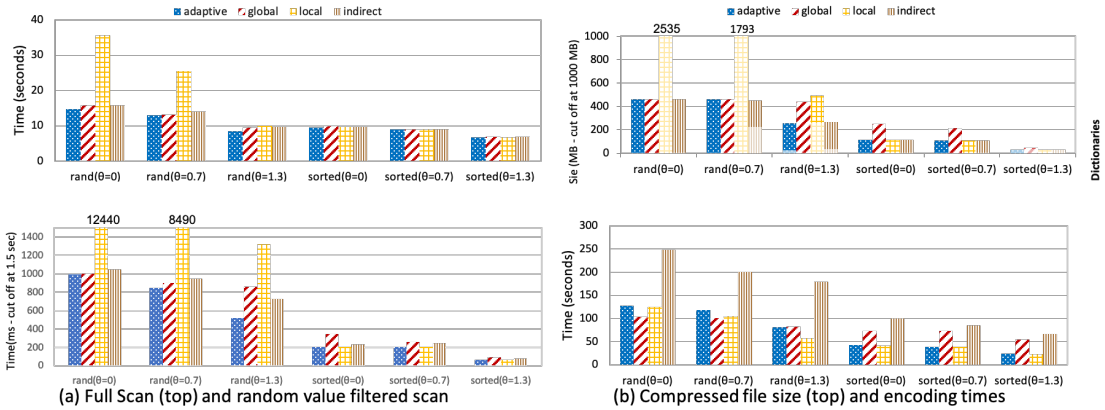

Figure 5: Storage and scan efficiency for various data distributions (70K distinct values)

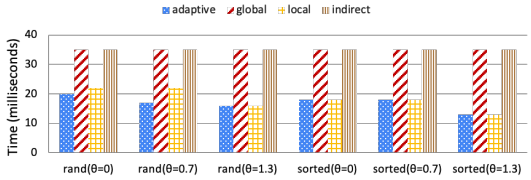

Figure 6: Random access

offsets is small, the main overhead is the extra seek to the 1st page. *iii)* The offset exists in the differential dictionary of the current page. This is a more rare case, in which we are in a small differential page (∼100 values) and we do not need to access another page so that I/O is minimal (2msecs). As shown, the gain in second and third case is higher than what we lose in the first case. Obviously, average random access time depends on the distribution and how many times a random access query is assigned to each of these cases.

4.2 Cost function evaluation

We use tables from Yelp, EDGAR Log File Dataset, TPC-DS and MAG to evaluate the cost function. Yelp's table contains two attributes. The first is a string identifier of users with at least one review and the second contains their friends. We compress the first attribute (1.9GB) which consists of 1.6M distinct ids. Edgar's table contains 23.7M records with an attribute with 78181 distinct IPs (356MB). We are using 'sales' table from TPC-DS in which we replace the customer's identifiers with their alternative string identifiers. This column contains 500K distinct ids (3.7GB). We also use the random distribution with skewness parameter $\theta = 1.3$ from the previous experiment. We compare local dictionaries, full differential dictionaries, the cost function and 3 other ways to select between local and differential: *i)* Offset. A local dictionary is stored when the offsets size increases. *ii)* Cumulative dictionary. A local dictionary is stored when the size of the different values in sequence exceeds a threshold (8MB). *iii)* Current total. The selection is done comparing the current total size required to store a local or a differential page. The results are shown in Figure 7. As shown, for Yelp, the cost function does not achieve the higher compression which is achieved with fully differential encoding. However, since the attribute contains 1.6M distinct values, this increases a lot the execution time. The cost function achieves higher compression than the other candidates while the compression time overhead is proportionally smaller. In the other datasets, the cost function compresses more efficiently with small time overheads. In TPC-DS, the cost function behaves like differential encoding, in EDGAR it compresses better than local/differential behaving the same with the other 3 alternatives, and in the synthetic dataset it achieves higher compression than local/differential by storing local dictionaries when the offset size increases. For clarity purposes, in Yelp, TPC-DS and EDGAR datasets, indirect coding does not store mappings and behaves the same as global encoding, which produce same sizes in similar times as fully differential.

4.3 Page omittance in EDGAR log file dataset

We used EDGAR log file to evaluate page omittance. The table consists of 362 pages (2^{16} records per page). Table 2 shows the results when searching for IP '98.6.98.djf' (1 occurrence). The table presents total time including I/O and scan costs, I/O times, and count of omitted pages or offset scan with page skipping enabled or not. Zone-maps filter out 10 pages while differential min/max omit 172 pages. Local and differential dictionaries skip offset scan when the value does not exist in the dictionary. As shown, with naive zone-maps the gain in I/O (skipping 10/362 pages) is small and there is no gain in the total time whereas using additional differential min/max the time decreases from 83 to 67 msecs.

format	time	I/O	omit pages/offset scan
global	138	119	10/0
global/no omit	137	120	0/0
local	170	156	10/351
local/no omit	173	160	0/361
diff	67	64	172/188
diff/no omit	83	80	0/360

Table 2: Page omittance for value '98.6.98.djf'

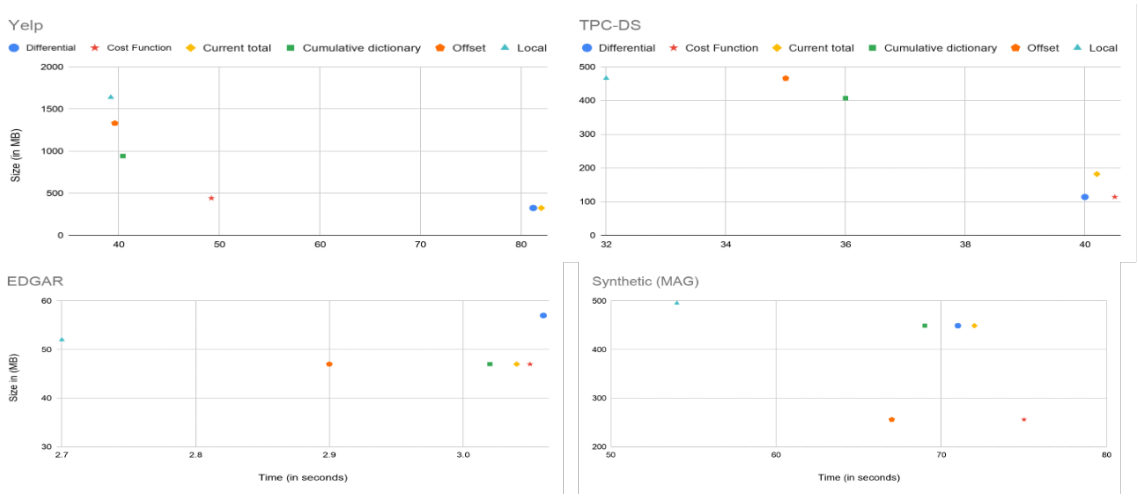

Figure 7: Compression size and encoding time for different encoding strategies

Table 3 shows the results when searching for IP '101.81.231.jde'. This is a value with higher selectivity, since there are 4808 occurrences. Zone maps do not omit any page, whereas differential min/max omit 36 pages and avoid offsets scan in 293 pages after dictionary look ups.

format	time	I/O	omit pages/offset scan
global	138	120	0/0
global/no omit	137	120	0/0
local	170	154	0/337
local/no omit	169	154	0/337
diff	81	77	36/293
diff/no omit	84	80	0/329

Table 3: Page omittance for value '101.81.231.jde'

4.4 Parallel Scan

We run this experiment on the OpenAIRE's citations table (330M citations for 31M publication DOIs). We split the table in 100 parts stored with Dask Parquet and enhanced Parquet (adaptive dictionaries). Finally, we ran an equi filtered scan on each partition. We ran the same experiment with our C++ implementation of local and adaptive encoding. The results of the slowest partition are shown in Figure 8.

As shown, the enhanced Parquet achieves faster scans in the examined dataset. What is also shown in the results is that the difference in scan times is much bigger when using C++. One of the main features of differential dictionary encoding is that it contains less string values and eliminates string comparisons during a scan. Specifically, it replaces string comparisons with offsets (integer) comparisons. Python interns strings so that a string comparison is almost equal to a pointer comparison. C++ does not do this at run time, so by reducing the number of string comparisons the gain is much greater. Given that, the benefit of using adaptive dictionaries is much greater in C++ (equi filter in 20ms).

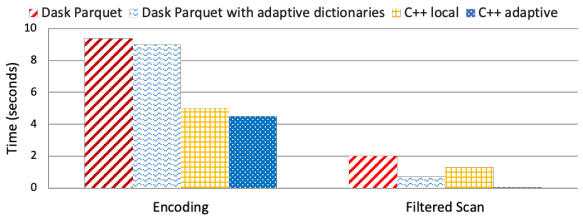

Figure 8: Parallel Scan Execution time

5 ACKNOWLEDGEMENTS

This work is supported by Horizon 2020 projects Human Brain Project and OpenAIRE-Advance with grant agreement numbers 945539 and 777541. The authors also thank Dr. Harry Dimitropoulos for his helpful comments.

6 CONCLUSIONS AND FUTURE WORK

We presented adaptive dictionaries, a compression technique that balances between local and global dictionaries using a cost function which adapts to changes in distribution. Using adaptive dictionaries, both the advantages of global and local encodings are utilized. In particular, adaptive dictionaries are created incrementally (per page), while each dictionary is sorted to allow efficient compression. Even if there are too many distinct values in the table, adaptive encoding is enabled in specific parts according to the cost function. Differential dictionaries are smaller (or empty) than global/local fitting better in cache and allowing fast look-ups. Adaptive dictionaries can omit more pages during a filtered scan than zone-maps. They avoid scanning the offsets until the first occurrence of the value and unnecessary dictionary look-ups if the value has already been found in a previous dictionary.

Future work concentrates on adapting the cost function depending on the workload and execution parameters. Offsets compression is also in our future plans. Finally, we will evaluate adaptive dictionaries with various page sizes.

REFERENCES

[1] https://google.github.io/snappy/. [Online; accessed 15-June-2020].

[2] https://github.com/facebook/zstd. [Online; accessed 15-June-2020].

[3] http://www.zlib.net/. [Online; accessed 15-June-2020].

[4] https://github.com/dask/fastparquet. [Online; accessed 15-June-2020].

[5] 5 ways Facebook improved compression at scale with Zstandard. https://engineering.fb.com/core-data/zstandard/. [Online; accessed 15-June-2020].

[6] Apache Parquet. https://parquet.apache.org/documentation/latest/. [Online; accessed 15-June-2020].

[7] Edgar Log File Data. http://www.sec.gov/dera/data/Public-EDGAR-log-file-data/2017/Qtr2/log20170630.zip. [Online; accessed 15-June-2020].

[8] Microsoft Academic Graph. https://docs.microsoft.com/en-us/academic-services/graph/reference-data-schema. [Online; accessed 15-June-2020].

[9] OpenAIRE API. https://api.openaire.eu. [Online; accessed 15-June-2020].

[10] ORC Creation Best Practices. https://community.cloudera.com/t5/Community-Articles/ORC-Creation-Best-Practices/ta-p/248963. [Online; accessed 15-June-2020].

[11] TPC-DS dataset. http://www.tpc.org/tpcds/. [Online; accessed 15-June-2020].

[12] Yelp dataset. https://www.yelp.com/dataset/download. [Online; accessed 15-June-2020].

[13] D. Abadi et al. Integrating compression and execution in column-oriented database systems. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, pages 671–682. ACM, 2006.

[14] D. J. Abadi, P. A. Boncz, and S. Harizopoulos. Column-oriented database systems. *Proceedings of the VLDB Endowment*, 2(2):1664–1665, 2009.

[15] A. Ailamaki, D. J. DeWitt, et al. Weaving Relations for Cache Performance. In *Proceedings of the VLDB Endowment*, volume 1, pages 169–180, 2001.

[16] R. Barber et al. In-memory BLU acceleration in IBM's DB2 and dashDB: Optimized for modern workloads and hardware architectures. In *Data Engineering (ICDE), 2015 IEEE 31st International Conference on*, pages 1246–1252. IEEE, 2015.

[17] B. Bhattacharjee et al. Efficient index compression in DB2 LUW. *Proceedings of the VLDB Endowment*, 2(2):1462–1473, 2009.

[18] C. Binnig et al. Dictionary-based order-preserving string compression for main memory column stores. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*, pages 283–296. ACM, 2009.

[19] R. Binns. Dask: Scalable Analytics in Python. https://dask.org. Accessed: 2020-11-20.

[20] I. K. Center. Impact of classic table reorganization on table-level compression dictionaries. https://www.ibm.com/support/knowledgecenter/en/SSEPGG_10.1.0/com.ibm.db2.luw.admin.dbobj.doc/doc/c0056502.html. [Online; accessed 15-June-2020].

[21] A. Floratou et al. Sql-on-hadoop: Full circle back to shared-nothing database architectures. *Proceedings of the VLDB Endowment*, 7(12):1295–1306, 2014.

[22] Y. He et al. RCFile: A fast and space-efficient data placement structure in MapReduce-based warehouse systems. In *Data Engineering (ICDE), 2011 IEEE 27th International Conference on*, pages 1199–1208. IEEE, 2011.

[23] Y. Huai et al. Major technical advancements in Apache HIVE. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of data*, pages 1235–1246. ACM, 2014.

[24] IBM. Optimize storage with deep compression in DB2 10. https://www.ibm.com/developerworks/data/library/techarticle/dm-1205db210compression/index.html. [Online; accessed 15-June-2020].

[25] J. Kestelyn. Introducing Parquet: Efficient Columnar Storage for Apache Hadoop. *Cloudera Blog (13th Mar. 2013).https://blog.cloudera.com/blog/2013/03/introducing-parquet-columnar-storage-for-apache-hadoop/*, 3, 2013.

[26] D. E. Knuth. *The art of computer programming: Volume 3: Sorting and Searching*. Addison-Wesley Professional, 1998.

[27] H. Lang, T. Mühlbauer, F. Funke, P. A. Boncz, T. Neumann, and A. Kemper. Data blocks: Hybrid oltp and olap on compressed storage using both vectorization and compilation. In *Proceedings of the 2016 International Conference on Management of Data*, pages 311–326. ACM, 2016.

[28] R. Lasch, I. Oukid, R. Dementiev, N. May, S. S. Demirsoy, and K.-U. Sattler. Fast & strong: The case of compressed string dictionaries on modern cpus. In *Proceedings of the 15th International Workshop on Data Management on New Hardware*, pages 1–10, 2019.

[29] C. Lemke, K.-U. Sattler, F. Faerber, and A. Zeier. Speeding up queries in column stores. In *International Conference on Data Warehousing and Knowledge Discovery*, pages 117–129. Springer, 2010.

[30] C. Liu, M. Umbenhower, H. Jiang, P. Subramaniam, J. Ma, and A. J. Elmore. Mostly order preserving dictionaries. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1214–1225. IEEE, 2019.

[31] B. M. Macq and J.-J. Quisquater. Cryptology for digital TV broadcasting. *Proceedings of the IEEE*, 83(6):944–957, 1995.

[32] G. Moerkotte. Small materialized aggregates: A light weight index structure for data warehousing. *Proceedings of the VLDB Endowment*, 1998.

[33] I. Müller, C. Ratsch, F. Faerber, et al. Adaptive string dictionary compression in in-memory column-store database systems. In *EDBT*, volume 14, pages 283–294, 2014.

[34] V. Raman et al. DB2 with BLU acceleration: So much more than just a column store. *Proceedings of the VLDB Endowment*, 6(11):1080–1091, 2013.

[35] V. Sikka et al. Efficient transaction processing in SAP HANA database: the end of a column store myth. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 731–742. ACM, 2012.

[36] I. H. Witten, A. Moffat, and T. C. Bell. *Managing Gigabytes (2nd Ed.): Compressing and Indexing Documents and Images*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1999.

[37] M. Zukowski, S. Heman, N. Nes, and P. Boncz. Super-scalar ram-cpu cache compression. In *22nd International Conference on Data Engineering (ICDE'06)*, pages 59–59. IEEE, 2006.